

Sistemas Operativos

1º Parcial 1C2025 – TM – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

- 1) a) La forma correcta es realizarlo a través de una syscall, le “pedimos” al SO que realice la operación por nosotros, realizando el correspondiente cambio de modo.
b) La forma incorrecta sería intentar ejecutar la instrucción privilegiada directamente. Esto fallaría ya que dicha instrucción sólo puede ser ejecutada en modo kernel. La CPU lanzaría una excepción, la cual sería atendida por el SO quien decidiría finalizar al proceso que la provocó.

- 2) Las condiciones necesarias para la ocurrencia de un deadlock son:

- Mutua exclusión
- Sin desalojo
- Retención y espera
- Espera circular

La prevención de deadlocks es una estrategia de tratamiento de deadlocks que evita la ocurrencia de estos a partir de la eliminación de al menos una de las condiciones enumeradas.

- 3) Cumple con los requisitos:

- Mutua exclusión: sólo un proceso puede adquirir el semáforo a la vez, el resto se quedarán bloqueados.
- Progreso: los únicos que pueden evitar que otro proceso adquiera el semáforo son los que también están realizando el wait.
- Espera limitada: los procesos se bloquean en una cola de dicho semáforo y luego se “despiertan” a medida que se hace signal al semáforo, respetando el orden de bloqueo.
- Velocidad de los procesos: la solución funciona de forma adecuada sin importar la velocidad de uso por parte de los procesos.

Los wait y signal operan sobre la variable semáforo que es compartida, por lo que existe una región crítica. Para que el acceso a la misma sea atómica el SO podría usar una técnica de HW como por ejemplo testAndSet, o incluso desactivar interrupciones.

- 4)

a) Falso. Si bien al matar el proceso sus hilos finalizan de forma obligada, este no es obligatoriamente el caso de los procesos hijos. En la mayoría de sistemas operativos (Linux por ejemplo) el proceso hijo es una entidad independiente que puede continuar su ejecución sin depender del padre.

b) Falso. Se encuentran las siguientes diferencias en la imagen de los procesos:

- Stack: valor de retorno del fork.
- PCB: valor del PID y PPID (PID del proceso padre).

5)

- a) Falso. Esta transición es posible con cualquier algoritmo con desalojo (ej: SJF con desalojo).
- b) Falso. Si bien VRR implementa una cola con mayor nivel de prioridad, los procesos utilizan un Q' dentro de la misma. Esto limita la cantidad de tiempo en la que estos procesos son planificados bajo un mayor nivel de prioridad, imposibilitando la existencia de starvation.

Práctica

- 1) a) El SO planifica con VRR $Q=3$, no puede ser RR ya que en el instante 10 vuelve a elegir a KC debido a estar en la cola prioritaria.
 b) La biblioteca de KA utiliza SJF sin desalojo e implementa jacketing. La biblioteca de KB utiliza prioridades fijas donde KB es más prioritario que KA, podría ser con o sin desalojo y no implementa jacketing.

- 2) **mutexPedidos**, **mutexValidados** = 1
turnoLuciano = 3, **turnoLeonardo** = 0
pedidosPendientes, **valPendientes** = 0
límitePedidos = 10

Usuario	Página web	Luciano	Leonardo
diseño=elegirDiseño() wait (límitePedidos) wait (mutexPedidos) agregar(diseño, pedidos) signal (mutexPedidos) signal (pedidosPendientes)	while(true) { wait (pedidosPendientes) wait (mutexPedidos) diseño = retirar(pedidos) signal (mutexPedidos) signal (límitePedidos) validar(diseño) wait (mutexValidados) agregar(diseño, validados) signal (mutexValidados) signal (valPendientes) }	while(true) { wait (turnoLuciano) wait (valPendientes) wait (mutexValidados) diseño = retirar(validados) signal (mutexValidados) imprimir(diseño) signal (turnoLeonardo) }	while(true) { wait (turnoLeonardo) x3 wait (valPendientes) wait (mutexValidados) diseño = retirar(validados) signal (mutexValidados) imprimir(diseño) signal (turnoLuciano) x 3 }

- 3) a)

P1	WAIT(A)		WAIT(B)								WAIT(C)		BLOQ		
P2							WAIT(C)		WAIT(B)		BLOQ				
P3					WAIT(B)		BLOQ								
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

- b)

